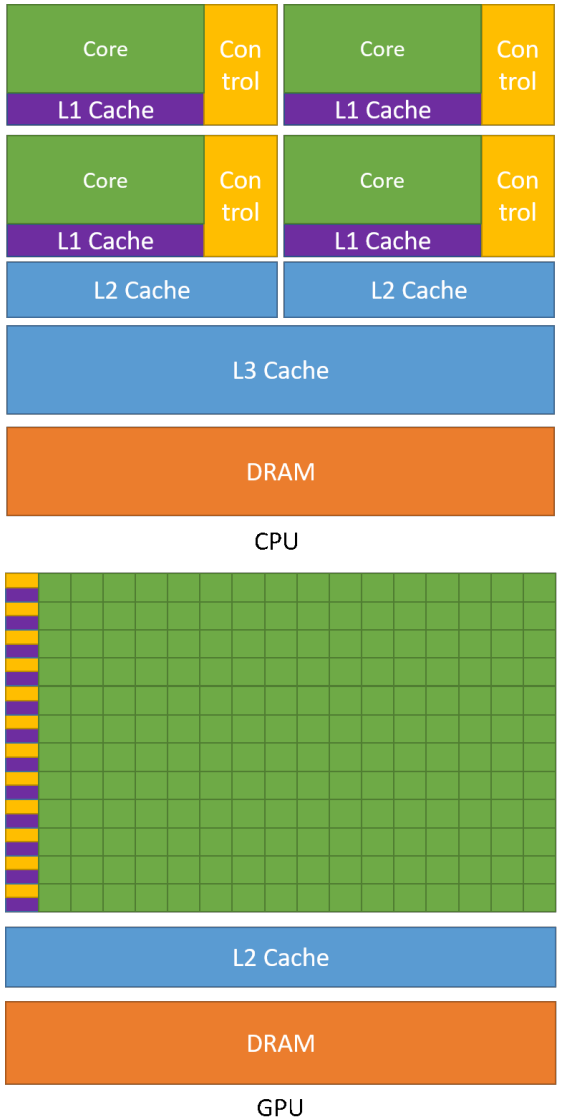
# GPU Architecture and Programming Models

Max Koch

# What is a GPU?

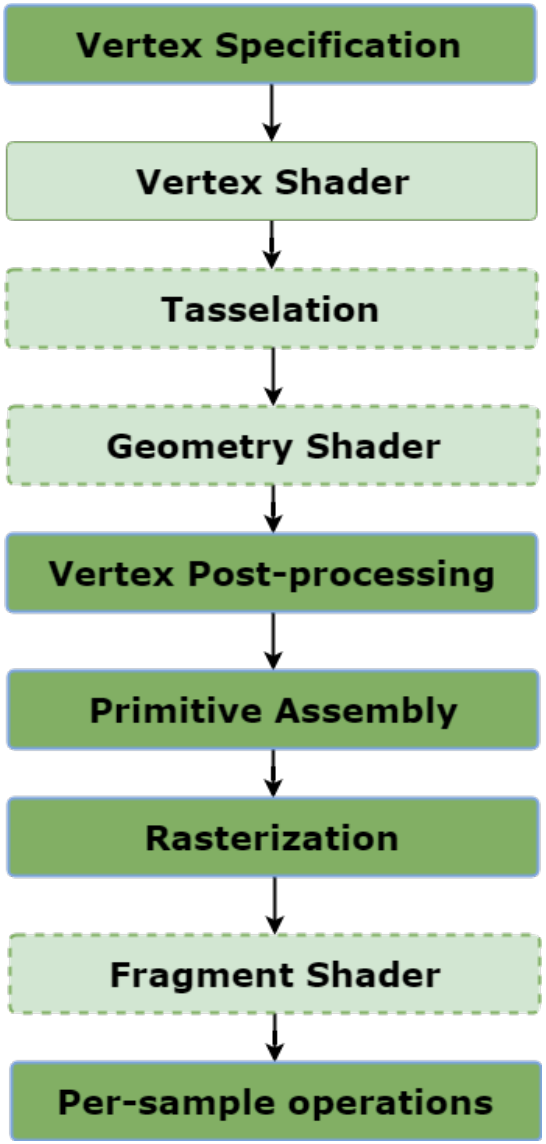
- **GPU** — Graphics Processing Unit
- Hardware accelerator for massively parallel workloads
- CPU: few, complex, out-of-order cores
- GPU: **thousands** of simple, in-order cores
- Originally designed for real-time 3D graphics rendering
- Today: ML training, scientific simulation, and other massively parallel workloads




The GPU Devotes More Transistors to Data Processing
From NVIDIA — CUDA Programming Guide

# GPU Use Case: Graphics Rendering

- Pre-2001: **fixed-function pipeline** — no programmability
- Then: **programmable shaders**
  - **Vertex shader**: transforms 3D geometry to screen space
  - **Fragment shader**: per-pixel color (lighting, textures)
- Same program runs on every vertex/pixel independently




OpenGL Rendering Pipeline
From GeeksforGeeks - OpenGL Rendering Pipeline | An Overview

# General-Purpose GPU (GPGPU)

- Programmable shaders can be repurposed for general-purpose computing
  - Disguise computations as rendering tasks
- NVIDIA introduced **CUDA** in 2007 to enable native general-purpose GPU programming

Now we can use our GPUs to generate images… but differently than before!


Generated with Gemini

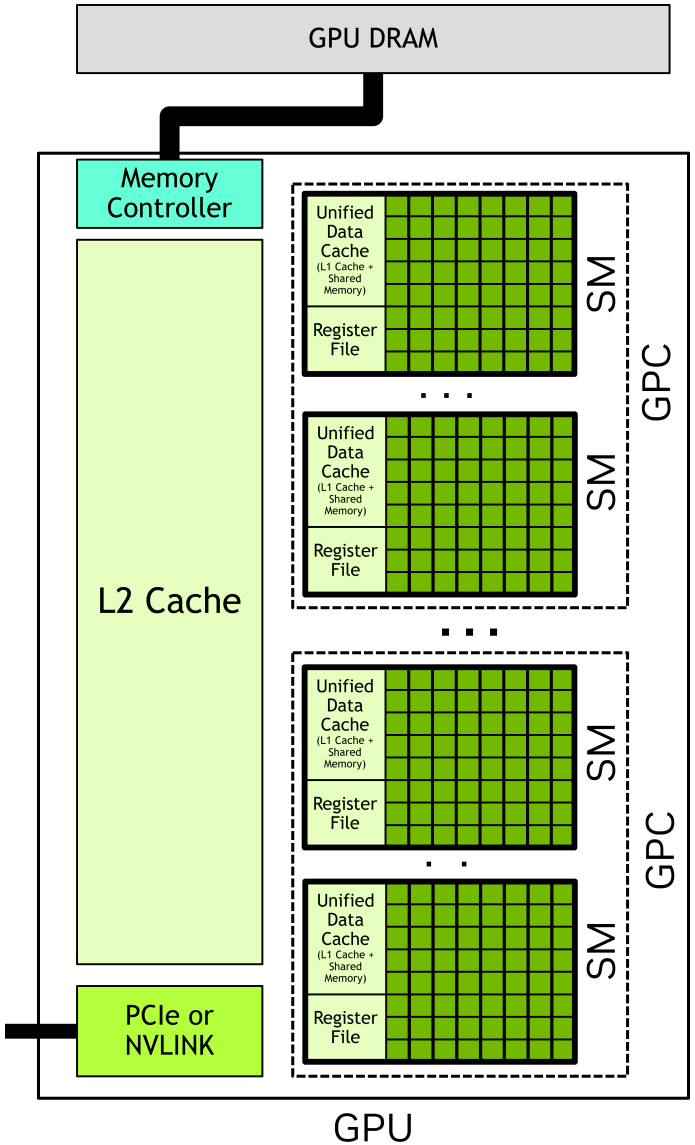
# NVIDIA DGX Spark

- SoC: NVIDIA GB10 Grace Blackwell Superchip
- 128GB of unified memory shared between CPU and GPU
- **The hardware we will use in this course**


flambe — NVIDIA DGX Spark with GB10 Grace Blackwell Superchip

# GPU Architecture Overview

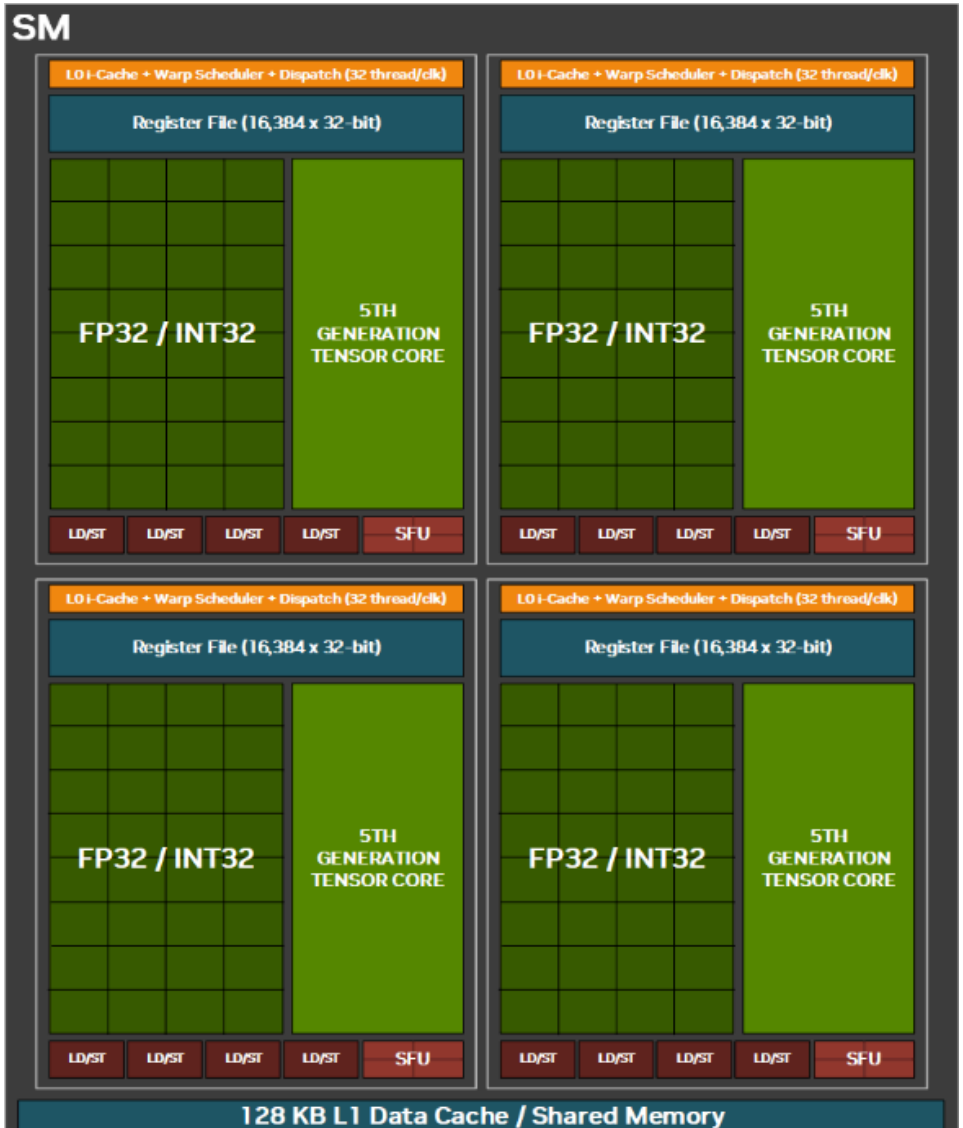
- GPUs have an array of **Streaming Multiprocessors (SMs)**
- Each SM is an independent parallel processor
- All SMs share an **L2 cache**
- Off-chip **HBM** (High Bandwidth Memory) — high bandwidth, high latency
- CPU and GPU communicate over PCIe or NVLink




GPU system with SM array, shared L2, and HBM
Adopted from: NVIDIA — CUDA Programming Guide

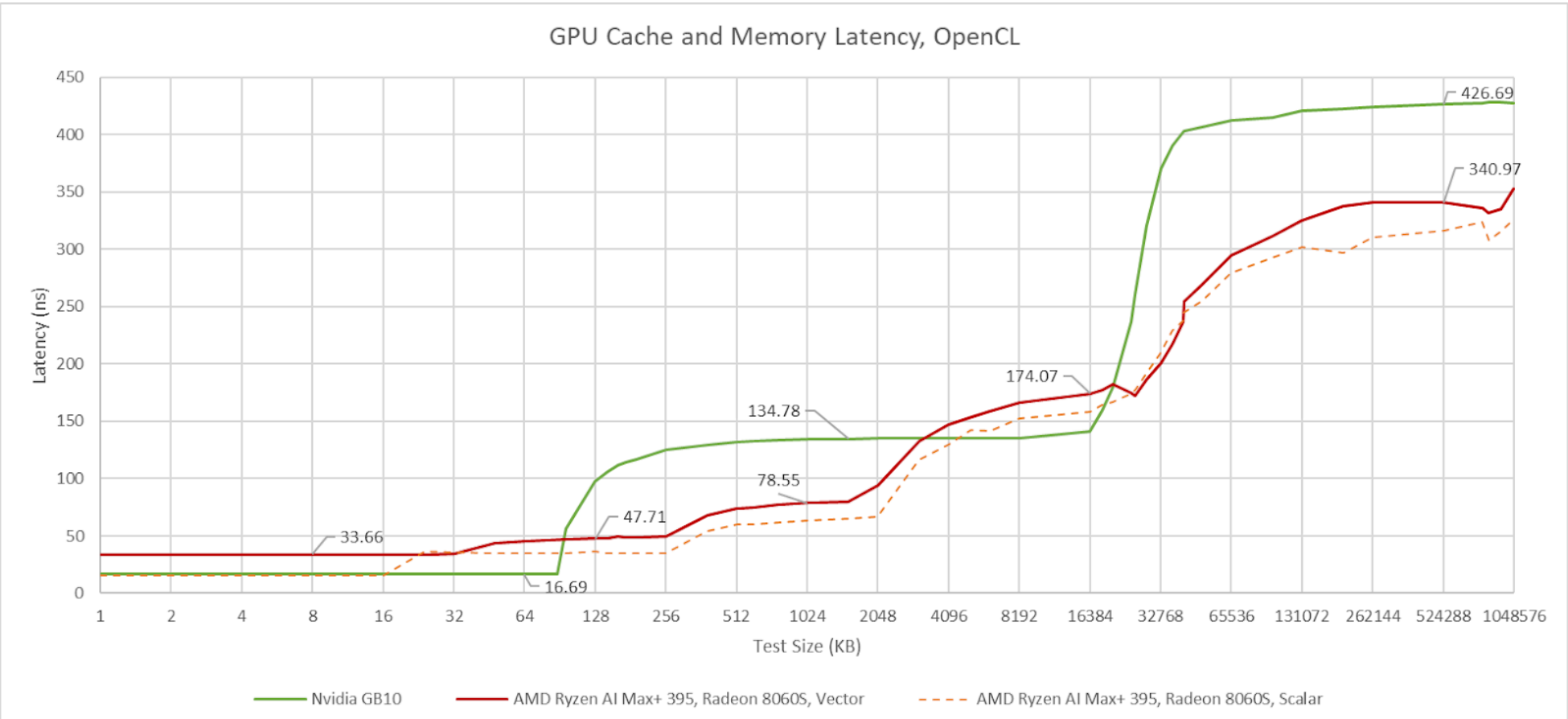
# Streaming Multiprocessor (SM)

- Each SM contains:
  - **CUDA cores**: contain ALUs for scalar integer and floating-point ops
  - **Tensor Cores**: hardware matrix multiply-accumulate (MMA)
  - **Shared memory / L1 cache**: fast on-chip scratchpad memory
  - **Register file**: large, per-thread
  - **Warp schedulers**: dispatch groups of 32 threads, fast context switching
- Warps execute in **SIMT**: single instruction, multiple threads




The Blackwell Streaming Multiprocessor (SM)
Adopted from: NVIDIA — NVIDIA RTX Blackwell GPU Architecture

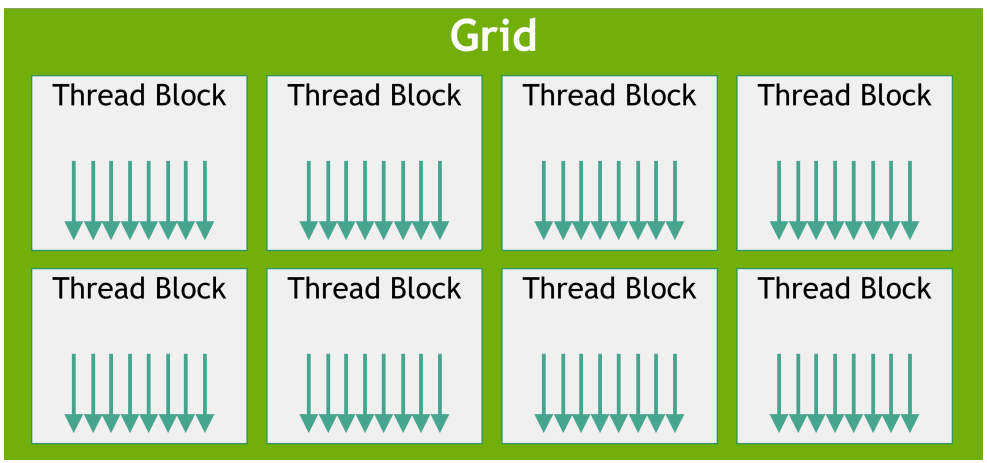
# GB10 Memory Latencies




From Chips and Cheese — Analyzing NVIDIA GB10's GPU

# The CUDA Programming Model

- **Thread:** single execution unit with private registers
- **Warp:** 32 threads executing in lockstep
  - If threads diverge (e.g. `if` statement), inactive threads are masked
- **Block** (Thread Block): group of warps with access to a common **shared memory**
- **Grid**: all blocks for one kernel launch
- Each block is assigned to one SM at runtime
- One SM can run multiple blocks concurrently




Grid of Thread Blocks
From NVIDIA — CUDA Programming Guide

# CUDA: Kernels and Shared Memory

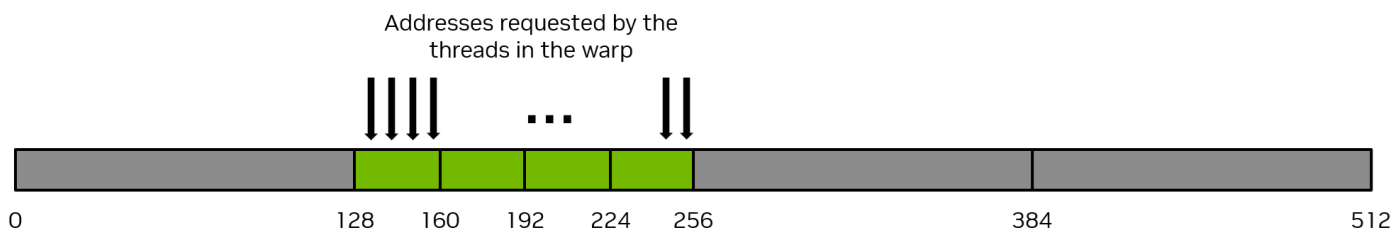
- `__global__`: marks function as GPU kernel
- `blockIdx`, `threadIdx`: built-in per-thread coordinates
- Shared memory is explicitly managed: declared with `__shared__`

```
 1 __global__ void transposeCoalesced(float *odata, const float *idata)
 2 {
 3   __shared__ float tile[TILE_DIM][TILE_DIM];
 4
 5   int x = blockIdx.x * TILE_DIM + threadIdx.x;
 6   int y = blockIdx.y * TILE_DIM + threadIdx.y;
 7   int width = gridDim.x * TILE_DIM;
 8
 9   // Load tile from global memory to shared memory
10   for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS)
11      tile[threadIdx.y+j][threadIdx.x] = idata[(y+j)*width + x];
12
13   __syncthreads();
14
15   x = blockIdx.y * TILE_DIM + threadIdx.x;  // transpose block offset
16   y = blockIdx.x * TILE_DIM + threadIdx.y;
17
18   for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS)
19      odata[(y+j)*width + x] = tile[threadIdx.x][threadIdx.y + j];
20 }
```

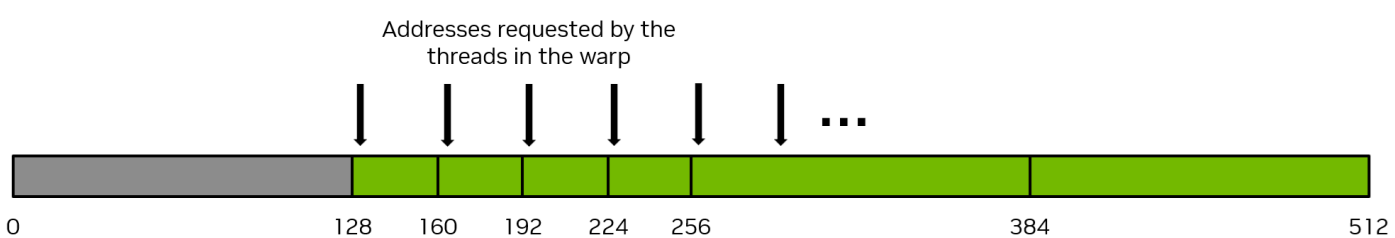
From NVIDIA — An Efficient Matrix Transpose in CUDA C/C++

# Memory Coalescing

- HBM is accessed in **32-byte segments**
- All 32 warp threads issue memory requests simultaneously
- **Coalesced**: 32 threads access 32 consecutive elements (1 transaction)




Coalesced memory access

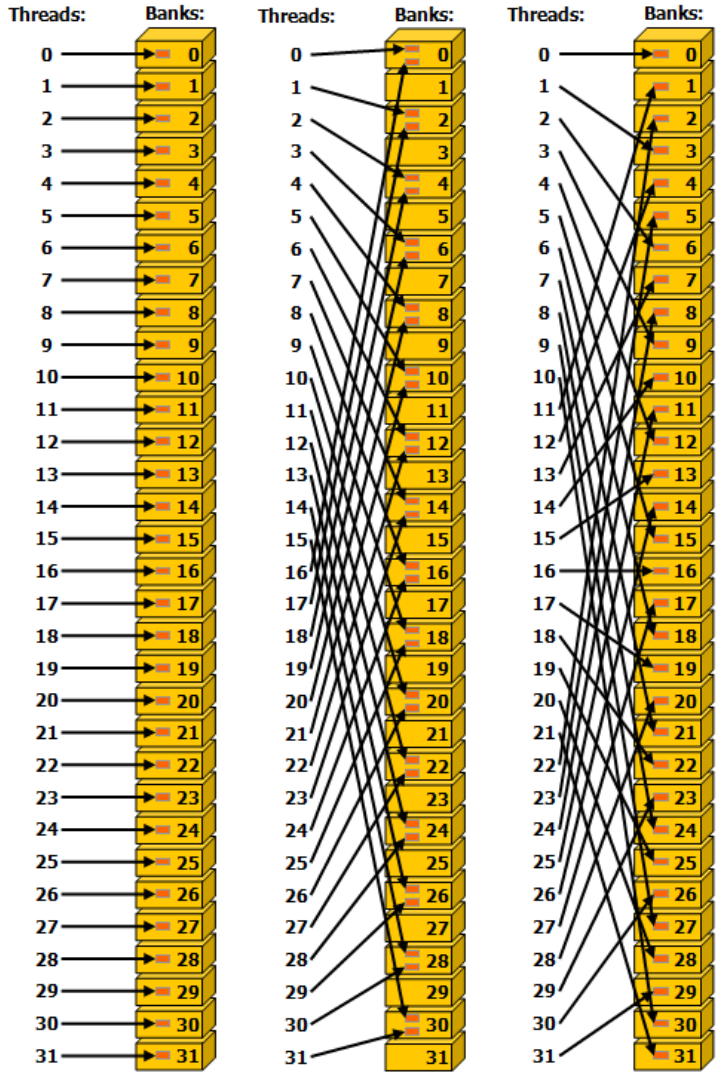



Uncoalesced memory access
Both from: NVIDIA — CUDA Programming Guide

# Bank Conflicts in Shared Memory

- Shared memory is divided into **32 banks** (4 bytes wide each)
- If different threads access **different addresses in the same bank**, the accesses are serialized → **bank conflict**

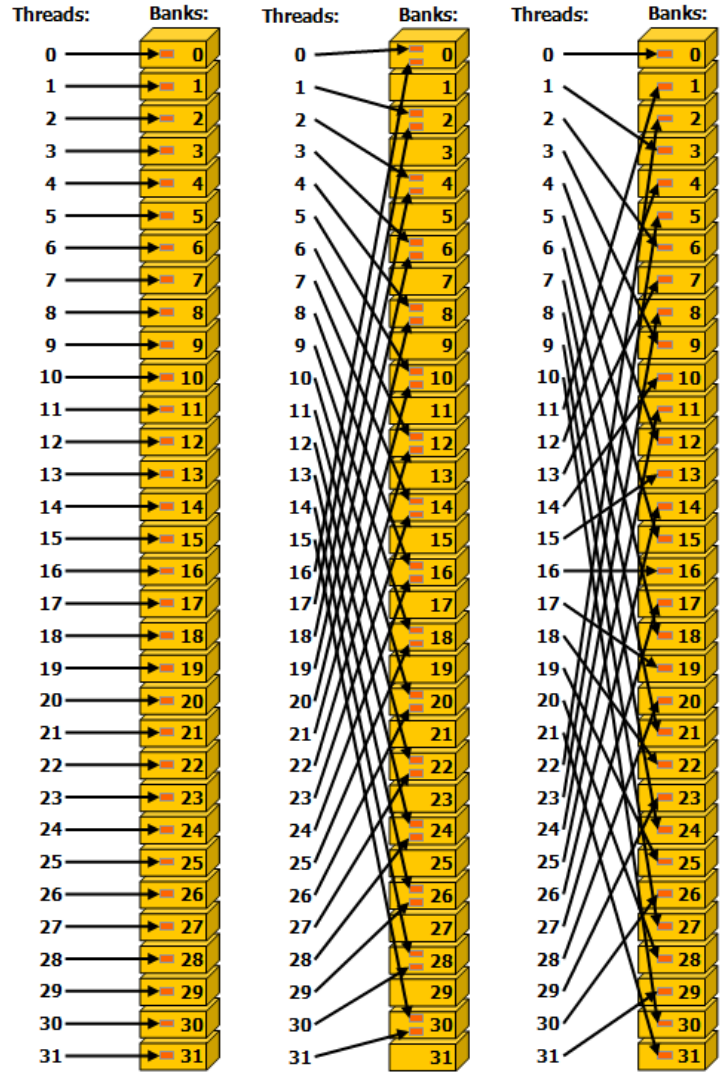
```
 1 // 4 bytes per element
 2 __shared__ float s[1024];
 3
 4 // initialize shared memory
 5 // ...
 6
 7 // all threads access different banks
 8 // no bank conflict
 9 val = s[threadIdx.x];
10
11 // all threads access the same bank
12 // bank conflict
13 val = s[threadIdx.x * 32];
```




Strided Shared Memory Accesses in 32 bit bank size mode.
From NVIDIA — CUDA Programming Guide

# Bank Conflict?

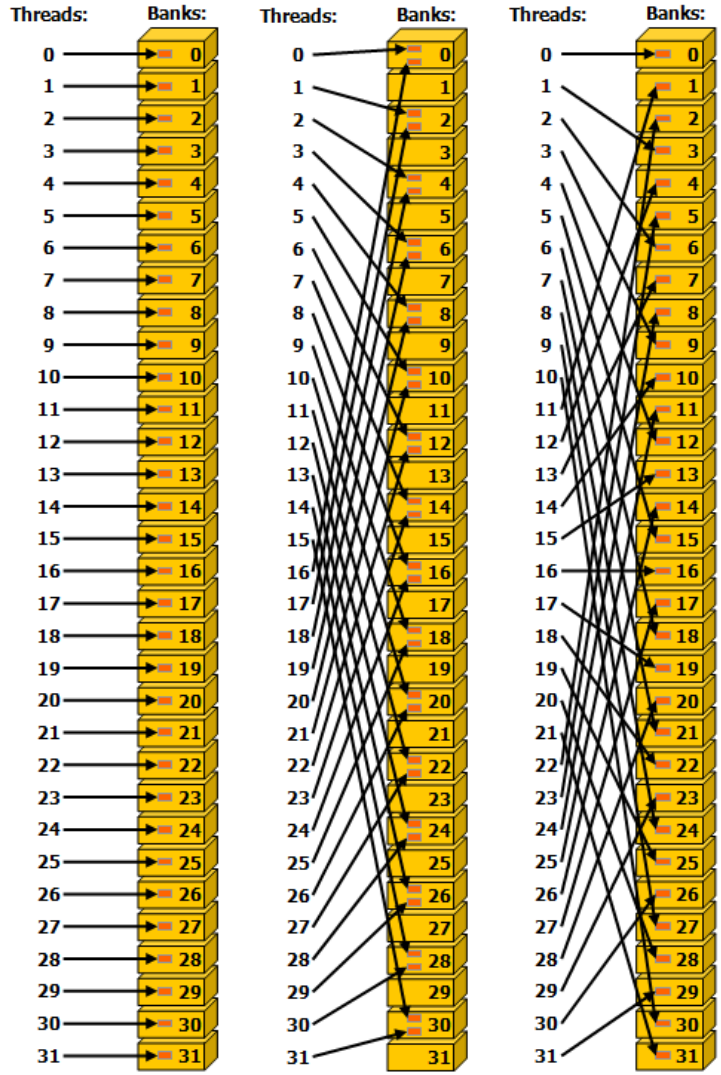
- **Left:**
- **Middle:**
- **Right:**




Strided Shared Memory Accesses in 32 bit bank size mode.
From NVIDIA — CUDA Programming Guide

# Bank Conflict?

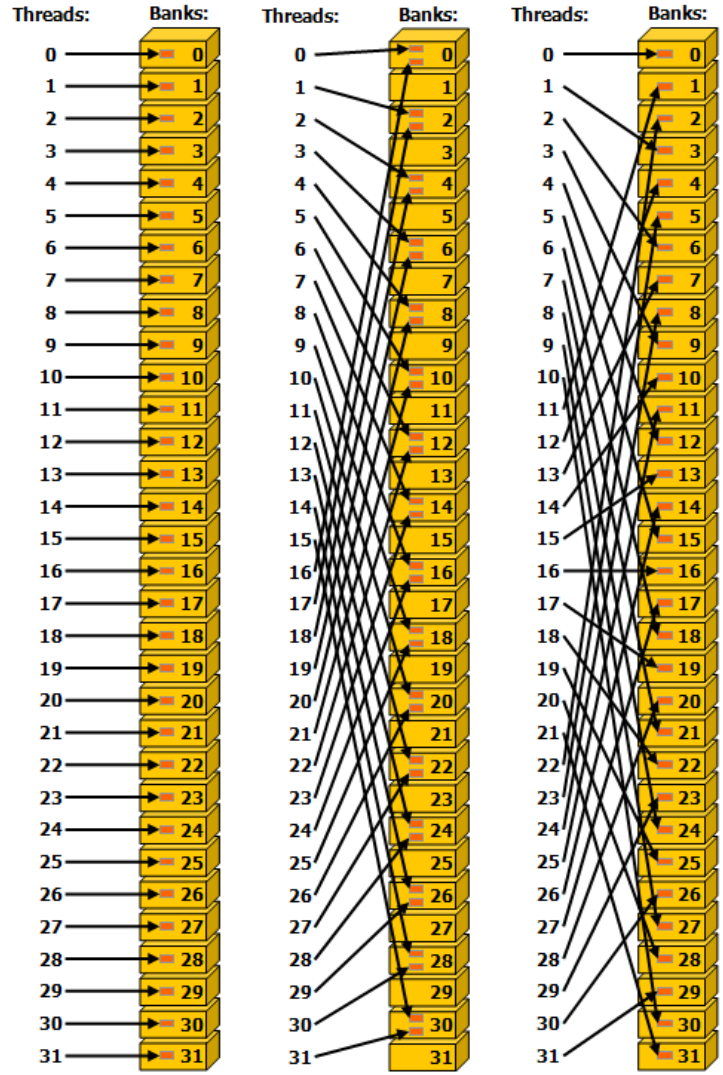
- **Left:** No bank conflict
- **Middle:**
- **Right:**




Strided Shared Memory Accesses in 32 bit bank size mode.
From NVIDIA — CUDA Programming Guide

# Bank Conflict?

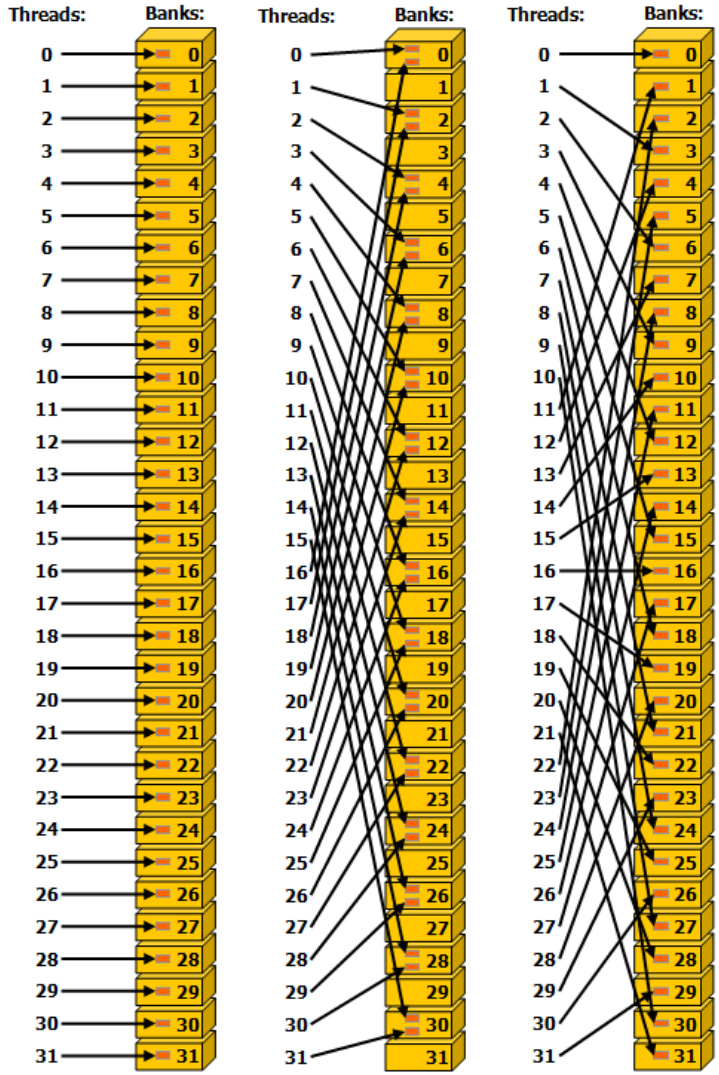
- **Left:** No bank conflict
- **Middle:** Bank conflict
- **Right:**




Strided Shared Memory Accesses in 32 bit bank size mode.
From NVIDIA — CUDA Programming Guide

# Bank Conflict?

- **Left:** No bank conflict
- **Middle:** Bank conflict
- **Right:** No bank conflict



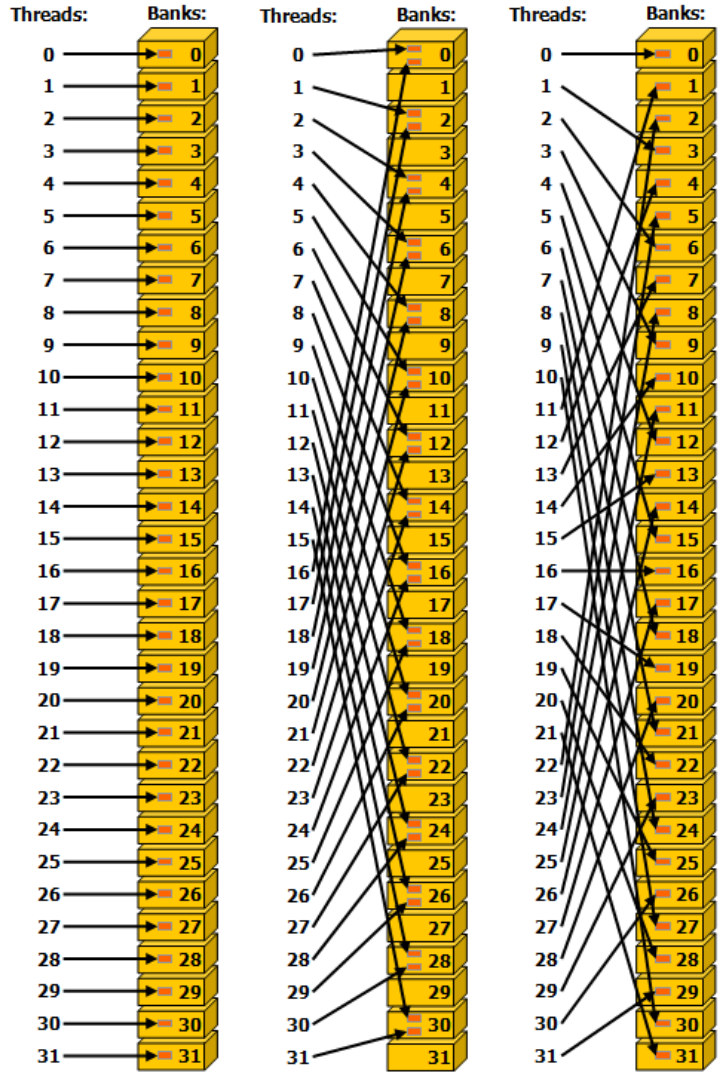

Strided Shared Memory Accesses in 32 bit bank size mode.
From NVIDIA — CUDA Programming Guide

# Bank Conflict?

- **Left:** No bank conflict
- **Middle:** Bank conflict
- **Right:** No bank conflict

## Middle:

```
1 // 4 bytes per element
2 __shared__ float s[1024];
3
4 // initialize shared memory
5 // ...
6
7 val = s[threadIdx.x * 2];
```




Strided Shared Memory Accesses in 32 bit bank size mode.
From NVIDIA — CUDA Programming Guide

# It’s getting complicated…

- Blackwell architecture offers:
  - **5th gen Tensor Cores**: FP4, FP6, FP8, FP16, BF16, TF32, FP64
  - **Tensor Memory Accelerator (TMA)**: hardware-managed async bulk copy to shared memory
  - **Thread Block Clusters**: SMs can share data via distributed shared memory
- But programming all this correctly and efficiently is increasingly complex with raw CUDA
- Interfaces between GPU generations are stable, but optimal performance often requires generation-specific tuning

# cuTile

- Released by NVIDIA in *late 2025*
- Python frontend for GPU programming via the **CUDA Tile programming model**
- cuTile kernels execute in parallel **on a logical grid of blocks**
- **Tile** is the unit of work
- **No threads, no explicit shared memory management**
- Abstracts away low-level details like **tensor core usage**

# CUDA Tile Programming Model

- Based on **arrays**, **tiles** and **operations on tiles**
- **Array:**
  - Tensor in global memory
  - Mutable via `ct.store()`
- **Tile:**
  - **Unit of work**
  - Tensor with no specified memory location
  - Dimension sizes must be powers of 2

# cuTile: Vector Addition

- Create vectors `vec_a`, `vec_b`, `vec_c` in GPU memory
- Define `grid` dimensions based on vector size and tile size
- Launch kernel with `ct.launch()`

```python
import torch
import cuda.tile as ct

vec_a = torch.randn(64, device='cuda')
vec_b = torch.randn(64, device='cuda')
vec_c = torch.empty_like(vec_a, device='cuda')

tile_size = 16

grid = (ct.cdiv(vec_a.size(0), tile_size), 1, 1)

# Launch kernel
ct.launch(torch.cuda.current_stream().cuda_stream,
        grid,  # 1D grid of processors
        vector_add,
        (vec_a, vec_b, vec_c, tile_size))
```

# cuTile: Vector Addition Kernel

- `@ct.kernel` decorator marks this function as cuTile kernel
- `pid = ct.bid(0)` gets the 1D block ID
- `ct.load()` loads a tile from global memory into a tile variable
- `ct.store()` writes a tile back to global memory

```
 1 @ct.kernel
 2 def vector_add(a, b, c, tile_size: ct.Constant[int]):
 3     # Get the 1D pid
 4     pid = ct.bid(0)
 5
 6     # Load input tiles
 7     a_tile = ct.load(a, index=(pid,), shape=(tile_size,))
 8     b_tile = ct.load(b, index=(pid,), shape=(tile_size,))
 9
10     # Perform elementwise addition
11     result = a_tile + b_tile
12
13     # Store result
14     ct.store(c, index=(pid, ), tile=result)
```

# `ct.load()`

- cuTile operators documentation: https://docs.nvidia.com/cuda/cutile-python/operations.html

## cuda.tile.load

```
cuda.tile.load(
    array,
    /,
    index,
    shape,
    *,
    order='C',
    padding_mode=PaddingMode.UNDETERMINED,
    latency=None,
    allow_tma=None,
)
```

Loads a tile from the *array* which is partitioned into a tile space.

The tile space is the result of partitioning the *array* into a grid of equally sized tiles specified by *shape*.

cuda.tile.load
From NVIDIA — cuTile Python

**Example**

```
t = ct.load(array, (i, j), (tm, tn))  # tile t has shape (tm, tn)
```